



MODULE IV



Module IV

- **Arduino for motor control- Control of DC Motor, Servo motor & Stepper Motor.**
- **Introduction to NodeMCU/ESP32, Overview of NodeMCU and its features, Programming NodeMCU via Arduino IDE,**
- **Interfacing LED, Gas Sensor,**
- **Introduction to Wi-Fi Connectivity with Node MCU**



Arduino for motor control

Interfacing different types of motors with the Arduino board (UNO) and how to connect the motor and drive it from your board.

There are three different type of motors –

- DC motor
- Servo motor
- Stepper motor

Arduino - DC Motor

Arduino - DC Motor

A DC motor (Direct Current motor) is the most common type of motor.

DC motors normally have just two leads, one positive and one negative.

If you connect these two leads directly to a battery, the motor will rotate.

If you switch the leads, the motor will rotate in the opposite direction.



Warning – Do not drive the motor directly from Arduino board pins. This may damage the board. Use a driver Circuit or an IC.

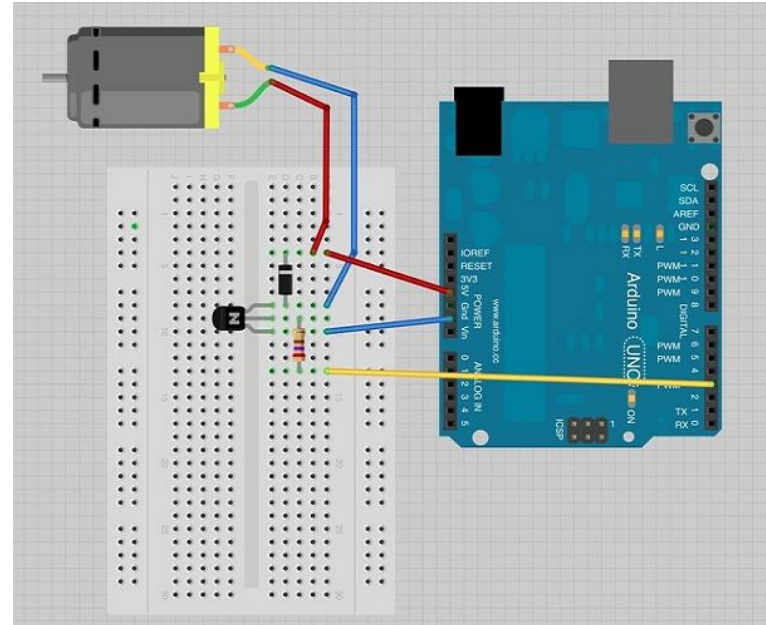
AIM :

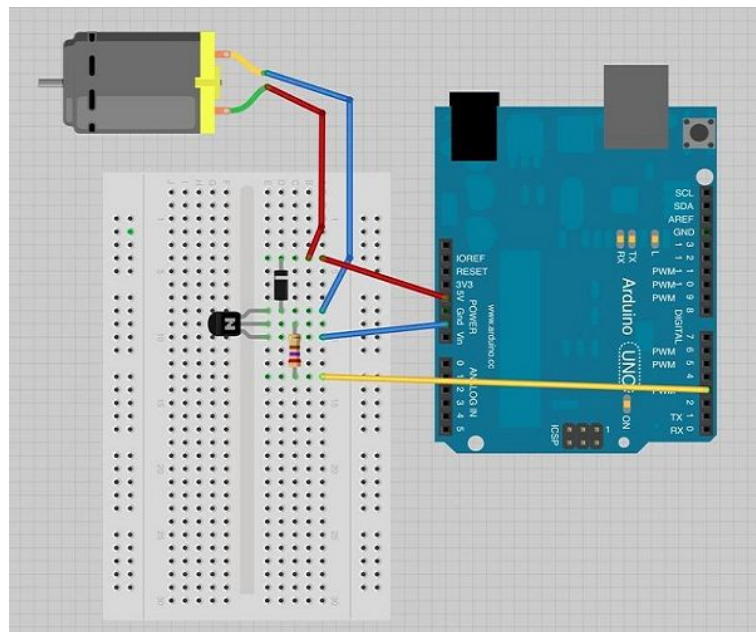
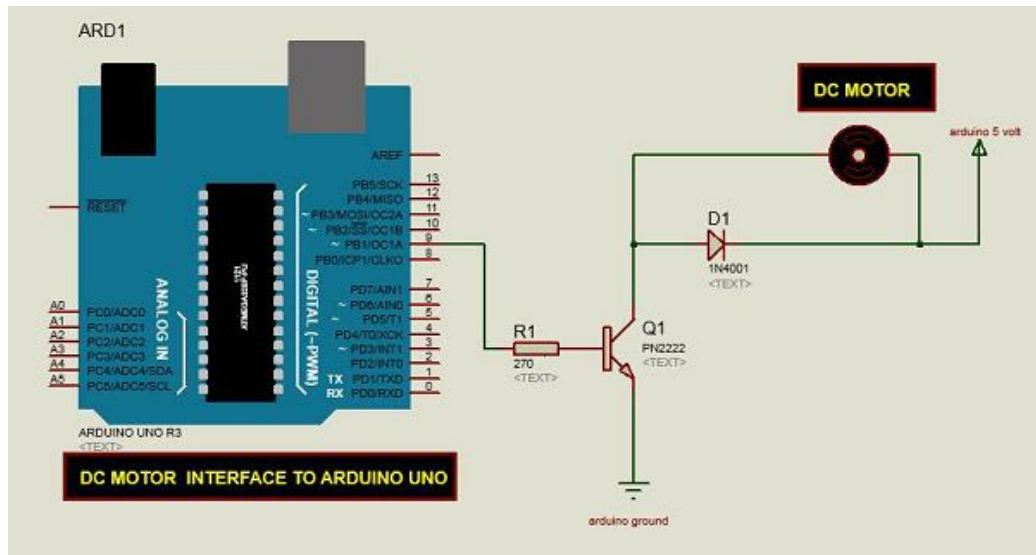
- Just make your motor spin
- Control motor speed
- Control the direction of the spin of DC motor

Components Required

You will need the following components –

- 1x Arduino UNO board
- 1x PN2222 Transistor
- 1x Small 6V DC Motor
- 1x 1N4001 diode
- 1x 270 Resistor







Why Do We Need a Transistor to Run a DC Motor with Arduino?

An **Arduino** pin can only give a small amount of current (about **40 mA max**), which is **not enough** to run a DC motor. A **transistor** works like an **electronic switch**. It allows us to control a **larger current** (from a battery or power supply) needed for the motor — using just a **small signal** from the Arduino.

Arduino controls the transistor.

Transistor controls the **motor power**.

This way, Arduino "tells" the transistor when to turn the motor on or off — **without getting damaged**.

What is a Forward-Biased Transistor?

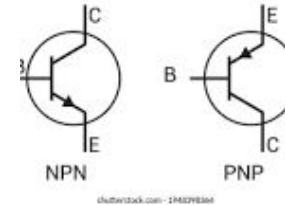
A **transistor** has three parts:

Base (B), **Collector (C)**, and **Emitter (E)**.

In order to **turn ON** a transistor and allow current to flow from **collector to emitter**, the transistor must be **forward biased**.

Forward Biasing (for NPN transistor):

- The **base** is connected to a **positive voltage** (through a resistor from Arduino).
- The **emitter** is connected to **ground (GND)**.
- This allows current to flow from **collector** → **emitter**, turning ON the circuit (like a motor or LED).





1. Spin Control : Just make your motor spin

```
int motorPin = 3;
void setup() {
}
void loop() {
  digitalWrite(motorPin, HIGH);
}
```

The transistor acts like a switch, controlling the power to the motor. Arduino pin 3 is used to turn the transistor on and off and is given the name 'motorPin' in the sketch.

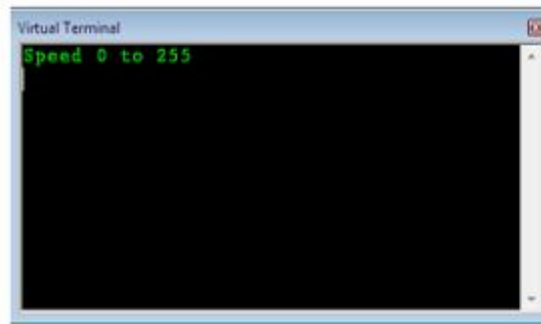
Motor will spin in full speed when the Arduino pin number 3 goes high.

2. Motor Speed Control

```
int motorPin = 9;

void setup() {
  pinMode(motorPin, OUTPUT);
  Serial.begin(9600);
  while (! Serial);
  Serial.println("Speed 0 to 255");
}

void loop() {
  if (Serial.available()) {
    int speed = Serial.parseInt();
    if (speed >= 0 && speed <= 255) {
      analogWrite(motorPin, speed);
    }
  }
}
```



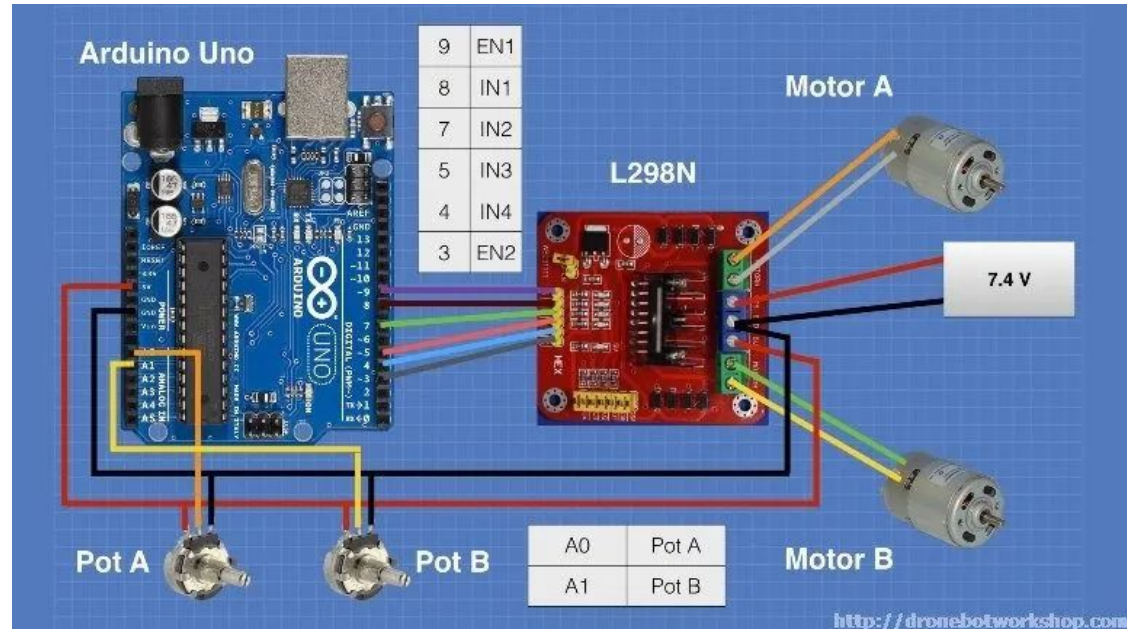


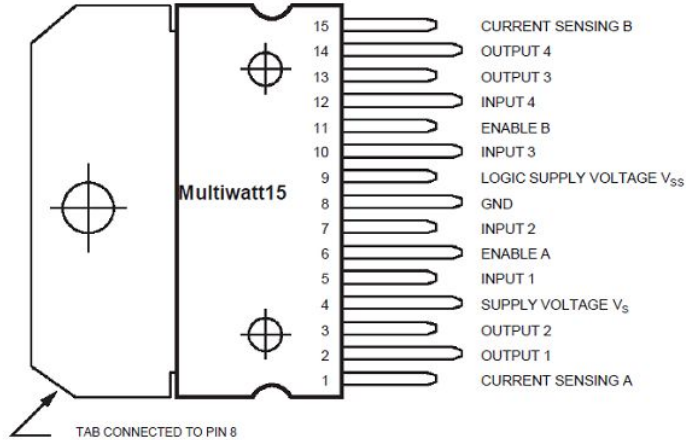
3. Spin Direction Control

To control the direction of the spin of DC motor, without interchanging the leads, you can use a circuit called an H-Bridge. An H-bridge is an electronic circuit that can drive the motor in both directions. H-bridges are used in many different applications. One of the most common application is to control motors in robots. It is called an H-bridge because it uses four transistors connected in such a way that the schematic diagram looks like an "H."

We will be using the L298 H-Bridge IC here. The L298 can control the speed and direction of DC motors and stepper motors, and can control two motors simultaneously. Its current rating is 2A for each motor. At these currents, however, you will need to use heat sinks.

Connecting 2 motors: using L298 H Bridge





There are three input pins for each motor,

Input1 (IN1), Input2 (IN2), and Enable1 (EN1) for Motor1

Input3, Input4, and Enable2 for Motor2.

We will be controlling only one motor in this example, we will connect the Arduino to IN1 (pin 5), IN2 (pin 7), and Enable1 (pin 6) of the L298 IC. Pins 5 and 7 are digital, i.e. ON or OFF inputs, while pin 6 needs a pulse-width modulated (PWM) signal to control the motor speed.



<u>IN1</u>	<u>IN2</u>	<u>Motor Behavior</u>
		BRAKE
1		FORWARD
	1	BACKWARD
1	1	BRAKE

Pin IN1 of the IC L298 is connected to pin 8 of Arduino while IN2 is connected to pin 9. These two digital pins of Arduino control the direction of the motor.

The EN A pin of IC is connected to the PWM pin 2 of Arduino. This will control the speed of the motor.

To set the values of Arduino pins 8 and 9, we have used the digitalWrite() function, and to set the value of pin 2, we have to use the analogWrite() function.



```
const int pwm = 2; //initializing pin 2 as pwm
const int in_1 = 8;
const int in_2 = 9;
//For providing logic to L298 IC to choose the direction of the DC motor

void setup() {
  pinMode(pwm,OUTPUT); //we have to set PWM pin as output
  pinMode(in_1,OUTPUT); //Logic pins are also set as output
  pinMode(in_2,OUTPUT);
}

void loop() {
  //For Clockwise motion , in_1 = High , in_2 = Low
  digitalWrite(in_1,HIGH);
  digitalWrite(in_2,LOW);
  analogWrite(pwm,255);
  /* setting pwm of the motor to 255 we can change the speed of rotation
  by changing pwm input but we are only using arduino so we are using highest
  value to driver the motor */
}
```

```
//Clockwise for 3 secs
delay(3000);
//For brake
digitalWrite(in_1,HIGH);
digitalWrite(in_2,HIGH);
delay(1000);
//For Anti Clockwise motion - IN_1 = LOW , IN_2 = HIGH
digitalWrite(in_1,LOW);
digitalWrite(in_2,HIGH);
delay(3000);
//For brake
digitalWrite(in_1,HIGH);
digitalWrite(in_2,HIGH);
delay(1000);
}
```

*Pin IN1 of the IC L298 is connected to pin 8 of Arduino while IN2 is connected to pin 9.
These two digital pins of Arduino control the direction of the motor.
The EN A pin of IC is connected to the PWM pin 2 of Arduino. This will control the speed of the motor.
The motor will run first in the clockwise (CW) direction for 3 seconds and then counter-clockwise (CCW) for 3 seconds.*

Arduino - Servo Motor

Arduino - Servo Motor

A Servo Motor is a small device that has an output shaft.

This shaft can be positioned to specific angular positions by sending the servo **a coded signal.**

As long as the coded signal exists on the input line, the servo will maintain the angular position of the shaft.

If the coded signal changes, the angular position of the shaft changes.

In practice, servos are used in radio-controlled airplanes to position control surfaces like the elevators and rudders. They are also used in radio-controlled cars, puppets, and of course, robots.



Servos are extremely useful in robotics.

The motors are small, have built-in control circuitry, and are extremely powerful for their size.

A standard servo such as the Futaba S-148 has 42 oz/inches of torque, which is strong for its size.

It also draws power proportional to the mechanical load. A lightly loaded servo, therefore, does not consume much energy.



Working of a Servo Motor

The servo motor has some control circuits and a potentiometer (a variable resistor, aka pot) connected to the output shaft.

This pot allows the control circuitry to monitor the current angle of the servo motor.

If the shaft is at the correct angle, then the motor shuts off.

If the circuit finds that the angle is not correct, it will turn the motor until it is at a desired angle.

The output shaft of the servo is capable of traveling somewhere around 180 degrees. Usually, it is somewhere in the 210-degree range, however, it varies depending on the manufacturer. A normal servo is used to control an angular motion of 0 to 180 degrees. It is mechanically not capable of turning any farther due to a mechanical stop built on to the main output gear.





The power applied to the motor is proportional to the distance it needs to travel.

So, if the shaft needs to turn a large distance, the motor will run at full speed.

If it needs to turn only a small amount, the motor will run at a slower speed. This is called **proportional control**.



How Do You Communicate the Angle at Which the Servo Should Turn?

The control wire is used to communicate the angle. The angle is determined by the duration of a pulse that is applied to the control wire. This is called **Pulse Coded Modulation**.

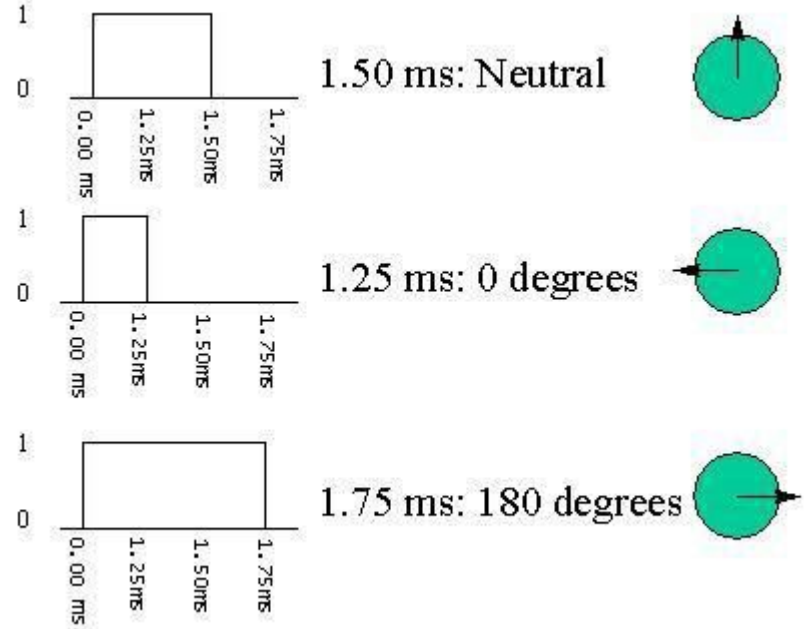
The servo expects to see a pulse every 20 milliseconds (.02 seconds).

The length of the pulse will determine how far the motor turns.

A 1.5 millisecond pulse, for example, will make the motor turn to the 90-degree position (often called as the neutral position).

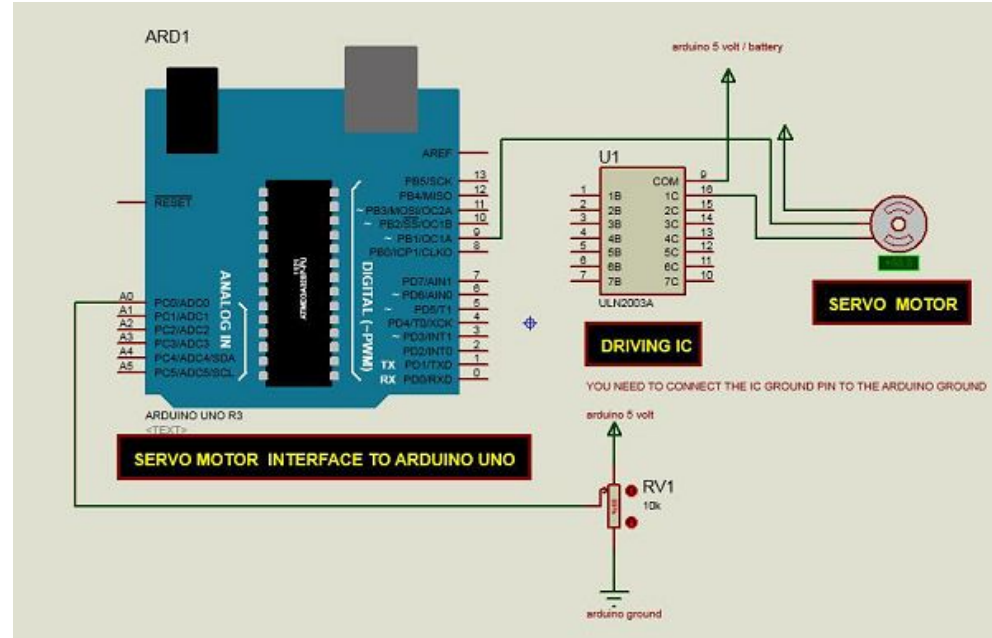
If the pulse is shorter than 1.5 milliseconds, then the motor will turn the shaft closer to 0 degrees.

If the pulse is longer than 1.5 milliseconds, the shaft turns closer to 180 degrees.



Procedure

- Servo motors have three terminals
 - **Power** - The power wire is typically red, and should be connected to the 5V pin on the Arduino.
 - **Ground** - The ground wire is typically black or brown and should be connected to one terminal of ULN2003 IC (10 -16).
 - **Signal** - To protect your Arduino board from damage, you will need some driver IC to do that. Here we have used ULN2003 IC to drive the servo motor. The signal pin is typically yellow or orange and should be connected to Arduino pin number 9.



Procedure

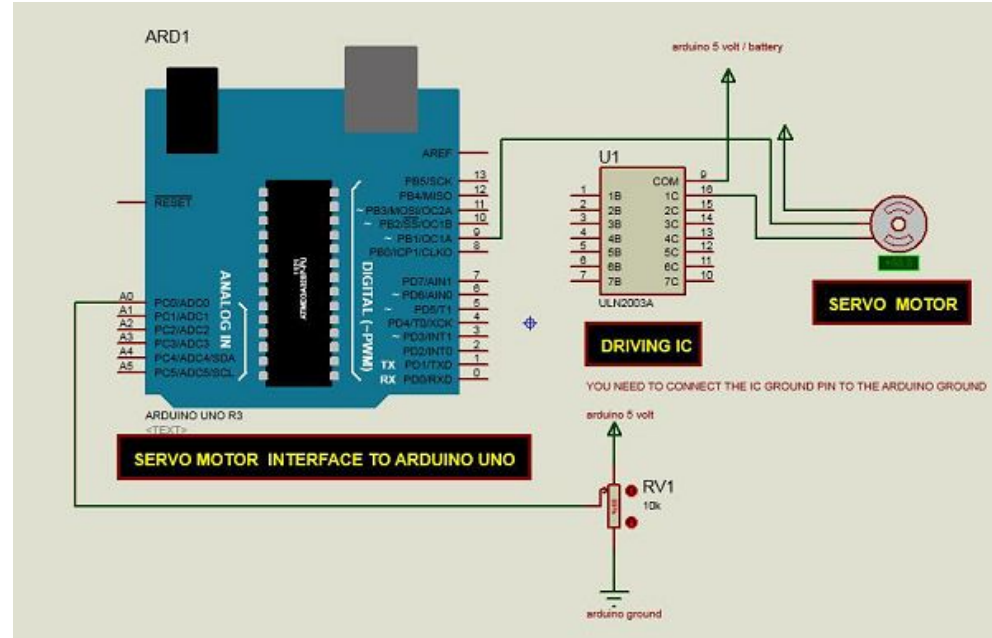
/* Controlling a servo position using a potentiometer (variable resistor) */

```
#include <Servo.h>
```

```
Servo myservo; // create servo object to control a servo
int potpin = 0; // analog pin used to connect the potentiometer
int val; // variable to read the value from the analog pin
```

```
void setup() {
  myservo.attach(9); // attaches the servo on pin 9 to the servo object
}
```

```
void loop() {
  val = analogRead(potpin);
  // reads the value of the potentiometer (value between 0 and 1023)
  val = map(val, 0, 1023, 0, 180);
  // scale it to use it with the servo (value between 0 and 180)
  myservo.write(val); // sets the servo position according to the scaled value
  delay(15);
}
```



Arduino - Stepper Motor

Stepper Motor



A Stepper Motor or a step motor is a brushless, synchronous motor, which divides a full rotation into a number of steps. Unlike a brushless DC motor, which rotates continuously when a fixed DC voltage is applied to it, a step motor rotates in discrete step angles.

The Stepper Motors therefore are manufactured with steps per revolution of 12, 24, 72, 144, 180, and 200, resulting in stepping angles of 30, 15, 5, 2.5, 2, and 1.8 degrees per step. The stepper motor can be controlled with or without feedback.

The motor spins very fast in one direction or another. You can vary the speed with the amount of power given to the motor, but you cannot tell the propeller to stop at a specific position.

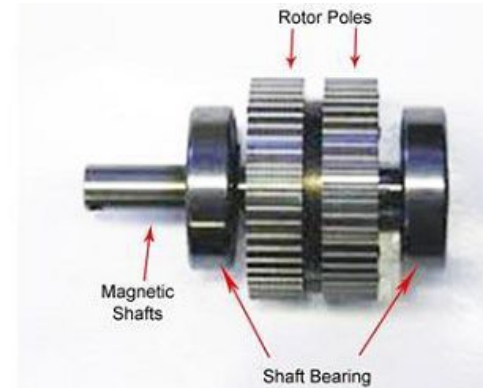
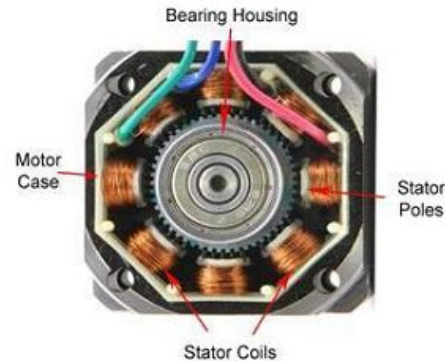


How a Stepper Motor Works?

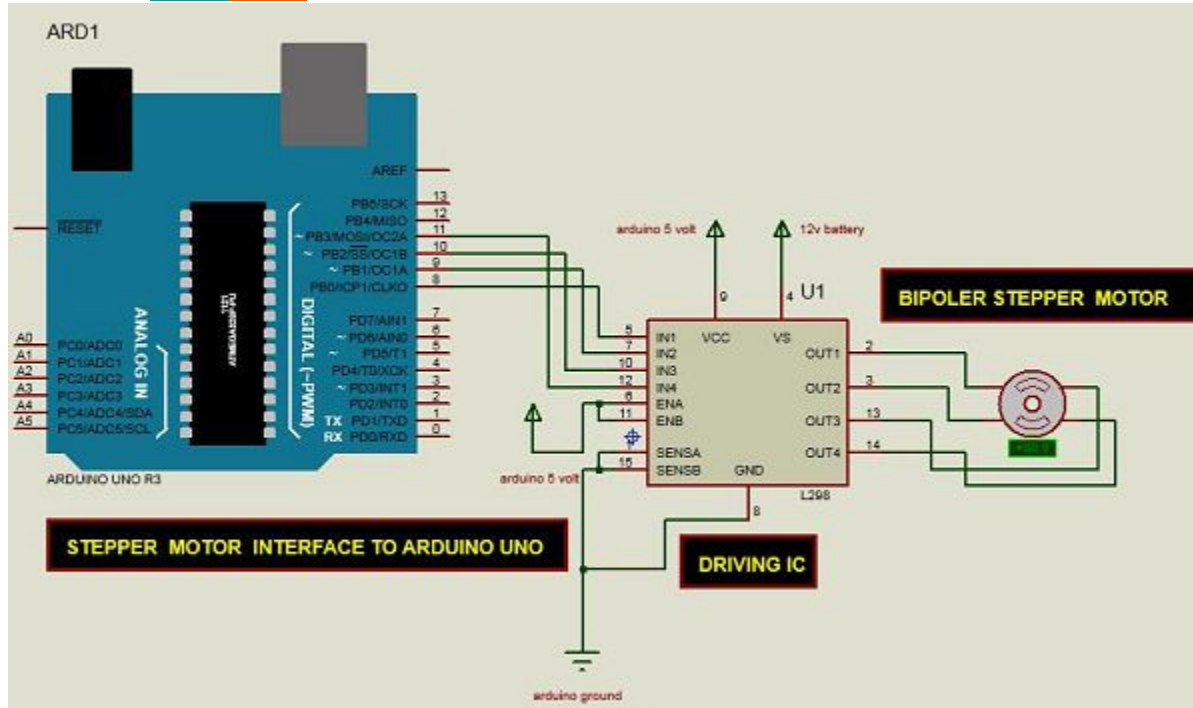
A regular DC motor spins in only direction whereas a Stepper motor can spin in precise increments.

Stepper motors can turn an exact amount of degrees (or steps) as desired. This gives you total control over the motor, allowing you to move it to an exact location and hold that position. It does so by powering the coils inside the motor for very short periods of time. The disadvantage is that you have to power the motor all the time to keep it in the position that you desire.

All you need to know for now is that, to move a stepper motor, you tell it to move a certain number of steps in one direction or the other, and tell it the speed at which to step in that direction. There are numerous varieties of stepper motors. The methods described here can be used to infer how to use other motors and drivers which are not mentioned in this tutorial. However, it is always recommended that you consult the datasheets and guides of the motors and drivers specific to the models you have.



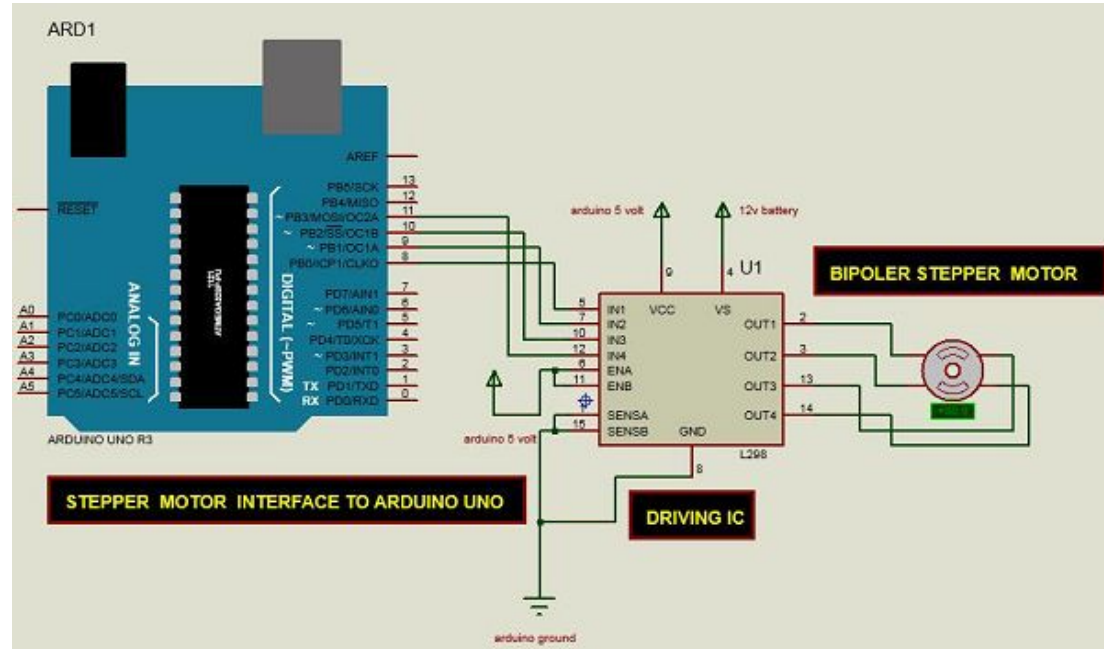
Procedure



Arduino Code

```
/* Stepper Motor Control */  
#include <Stepper.h>  
const int stepsPerRevolution = 90;  
// change this to fit the number of steps per revolution  
// initialize the stepper library on pins 8 through 11:  
Stepper myStepper(stepsPerRevolution, 8, 9, 10, 11);
```

```
void setup() {  
  // set the speed at 60 rpm:  
  myStepper.setSpeed(5);  
  // initialize the serial port:  
  Serial.begin(9600);  
}  
void loop() {  
  // step one revolution in one direction:  
  Serial.println("clockwise");  
  myStepper.step(stepsPerRevolution);  
  delay(500);  
  // step one revolution in the other direction:  
  Serial.println("counterclockwise");  
  myStepper.step(-stepsPerRevolution);  
  delay(500);  
}
```



This program drives a unipolar or bipolar stepper motor. The motor is attached to digital pins 8 - 11 of Arduino.



